ΜΑΘΗΜΑ 3 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.2

Λίστες και πλειάδες

Πολλές τιμές σε μία μεταβλητή

Θεωρία σε 15 μικρά βήματα + 18 ασκήσεις στο εργαστήριο Διάρκεια: 4 ώρες (3η–6η) 27 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 19/10/2026

Τι θα ξέρεις στο τέλος

1. Να φτιάχνεις μια **λίστα** και να παίρνεις ένα στοιχείο από τη **θέση** του.
2. Να **αλλάζεις**, να **προσθέτεις** και να **αφαιρείς** στοιχεία.
3. Να περνάς από όλα τα στοιχεία με **for** και να βρίσκεις άθροισμα, μέσο όρο, μέγιστο.
4. Να **γεμίζεις** μια λίστα με τιμές που δίνει ο χρήστης και να την **ταξινομείς**.
5. Να χρησιμοποιείς **πλειάδα** για τιμές που πάνε μαζί, π.χ. μια θέση (x, y) — θα τη χρειαστείς στο pygame.

Πώς δουλεύουμε

Η θεωρία είναι χωρισμένη σε **15 μικρά βήματα**. Κάθε βήμα έχει πάνω δεξιά την ετικέτα **ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ** με τον αριθμό της άσκησης στο site.
Διαβάζεις τη διαφάνεια → λύνεις την άσκηση → πας στο επόμενο βήμα.

Από το εύκολο στο δύσκολο

Ασκήσεις **1–7**: εύκολες, μία εντολή τη φορά.
Ασκήσεις **8–14**: μέτριες, συνδυάζουν 2–3 βήματα.
Ασκήσεις **15–17**: δύσκολες, όλα μαζί. Η **18** είναι για το σπίτι.

ΣΥΝΔΕΣΗ Το for της προηγούμενης φοράς περνούσε από αριθμούς (range) και από γράμματα. Σήμερα θα περνά από **λίστες**.

Θυμάσαι από την προηγούμενη φορά;

ΕΚΦΩΝΗΣΗ

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι εμφανίζει κάθε κομμάτι; Πρώτα σκέψου, μετά δες τη λύση.

ΛΥΣΗ

ΚΩΔΙΚΑΣ	ΑΠΟΤΕΛΕΣΜΑ	ΓΙΑΤΙ
`for i in range(2, 6): print(i)`	`2 3 4 5`	χωρίς το τέλος
`for ch in "ok": print(ch)`	`o k`	γράμμα-γράμμα
`len("Python")`	`6`	πόσα γράμματα
`"Python"[0]`	`"P"`	το πρώτο γράμμα
`"y" in "Python"`	`True`	υπάρχει μέσα

Γιατί αυτά

Ό,τι ξέρεις για τα κείμενα — **len**, **[0]**, **in**, **for** — δουλεύει **σχεδόν ίδιο** και στις λίστες. Ξεκινάμε με πλεονέκτημα.

ΠΑΡΑΤΗΡΗΣΗ Ένα κείμενο είναι μια σειρά από **γράμματα**. Μια λίστα είναι μια σειρά από **ό,τι θέλεις**: αριθμούς, λέξεις…

30 βαθμοί = 30 μεταβλητές;

ΧΩΡΙΣ ΛΙΣΤΑ

```
grade1 = 12
grade2 = 8.5
grade3 = 17
# ... άλλες 27 γραμμές ...
avg = (grade1 + grade2 + grade3 + ...) / 30
```

ΜΕ ΛΙΣΤΑ

```
grades = [12, 8.5, 17, 9, 15]

avg = sum(grades) / len(grades)
print(avg)
12.3
```

Τι πάει στραβά

Όσο περισσότεροι οι βαθμοί, τόσο μεγαλύτερος ο κώδικας. Κι αν τους δίνει ο χρήστης, **δεν ξέρεις** πόσες μεταβλητές να φτιάξεις.

Τι κερδίζεις

Ένα όνομα για όλες τις τιμές. Ο ίδιος κώδικας δουλεύει για 5, 30 ή 3.000 βαθμούς, χωρίς καμία αλλαγή.

Η ΙΔΕΑ Η λίστα είναι σαν ένα **κουτί με θήκες**: ένα όνομα, πολλές τιμές, η καθεμία στη θέση της.

Φτιάχνω μια λίστα

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 1

```
grades = [12, 8.5, 17]
names = ["Άννα", "Νίκος", "Ελένη"]
empty = []

print(names)
['Άννα', 'Νίκος', 'Ελένη']
print(len(names))
3
print(len(empty))
0
```

Πώς γράφεται

Ανοίγεις αγκύλη **[**, γράφεις τις τιμές **χωρισμένες με κόμμα**, κλείνεις με **]**. Τα κείμενα θέλουν εισαγωγικά, οι αριθμοί όχι.

Πόσα στοιχεία έχει;

Το **len(λίστα)** δίνει το πλήθος — όπως το len("Python") έδινε πόσα γράμματα έχει η λέξη.

Άδεια λίστα

Το **[]** είναι μια λίστα χωρίς τίποτα μέσα. Θα τη γεμίσουμε αργότερα (βήμα 10).

ΘΥΜΗΣΟΥ Αγκύλες **[ ]** = λίστα. Ένα στοιχείο ή εκατό, πάντα με **κόμμα** ανάμεσα.

Παίρνω ένα στοιχείο από τη θέση του

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 2

fruits = ["μήλο", "αχλάδι", "κεράσι", "σύκο"]

ΘΈΣΗ	0	1	2	3
τιμή	"μήλο"	"αχλάδι"	"κεράσι"	"σύκο"
από το τέλος	-4	-3	-2	-1

```
print(fruits[0])
μήλο
print(fruits[2])
κεράσι
print(fruits[-1])
σύκο
```

Το μέτρημα ξεκινά από το 0

Το **πρώτο** στοιχείο είναι στη θέση **0**, το δεύτερο στη **1**... Με 4 στοιχεία η τελευταία θέση είναι η **3**.

Το [-1] είναι το τελευταίο

Δεν χρειάζεται να ξέρεις πόσα στοιχεία έχει η λίστα: το **[-1]** είναι πάντα το τελευταίο, το **[-2]** το προτελευταίο.

ΣΥΧΝΟ ΛΑΘΟΣ **fruits[4]** → IndexError: ζήτησες θέση που **δεν υπάρχει**. Εδώ οι θέσεις είναι 0, 1, 2, 3.

Αλλάζω ένα στοιχείο

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 3

```
grades = [12, 8, 17]
grades[1] = 14      # ο 2ος βαθμός γίνεται 14
print(grades)
[12, 14, 17]
```

ΜΕ ΠΡΑΞΗ

```
grades[0] = grades[0] + 1   # +1 στον 1ο
print(grades)
[13, 14, 17]
```

Όπως μια μεταβλητή

Όπως γράφεις **x = 5**, γράφεις **grades[1] = 14**: «βάλε στη θέση 1 την τιμή 14». Η παλιά τιμή χάνεται.

Ποια θέση;

Οι θέσεις μετράνε από το 0. Ο **2ος** βαθμός είναι στη θέση **1**.

Στο κείμενο δεν γίνεται

Το word[0] = "M" σε κείμενο βγάζει σφάλμα. **Η λίστα αλλάζει, το κείμενο όχι.**

ΠΡΟΣΟΧΗ Αλλάζεις μόνο θέση που **υπάρχει**. Για νέο στοιχείο θέλεις **append** — επόμενο βήμα.

Προσθέτω στοιχεία

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 4

```
cart = ["ψωμί"]

cart.append("γάλα")       # στο τέλος
print(cart)
['ψωμί', 'γάλα']

cart.insert(0, "αυγά")    # στη θέση 0
print(cart)
['αυγά', 'ψωμί', 'γάλα']
```

append(τιμή)

Βάζει την τιμή **στο τέλος** της λίστας. Αυτό θα χρησιμοποιείς σχεδόν πάντα.

insert(θέση, τιμή)

Βάζει την τιμή **στη θέση** που λες. Τα υπόλοιπα πάνε μία θέση δεξιά.

Η τελεία

Γράφεις **όνομα_λίστας.append(...)**. Η λίστα αλλάζει αμέσως, δεν χρειάζεται «cart = ...».

ΣΥΧΝΟ ΛΑΘΟΣ **cart = cart.append("τυρί")** → χάνεις τη λίστα! Γράψε σκέτο cart.append("τυρί").

Αφαιρώ στοιχεία

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 5

```
cart = ["αυγά", "ψωμί", "γάλα", "τυρί"]

cart.remove("ψωμί")    # βγάζει την τιμή
print(cart)
['αυγά', 'γάλα', 'τυρί']

last = cart.pop()      # βγάζει το τελευταίο
print(last)
τυρί
print(cart)
['αυγά', 'γάλα']
```

remove(τιμή)

Ψάχνει την **τιμή** και τη βγάζει από τη λίστα (την πρώτη που θα βρει).

pop()

Βγάζει το **τελευταίο** στοιχείο και σου το δίνει, για να το κρατήσεις σε μεταβλητή. Το **pop(0)** βγάζει το πρώτο.

ΣΥΧΝΟ ΛΑΘΟΣ **cart.remove("μέλι")** όταν δεν υπάρχει μέλι → ValueError. Έλεγξε πρώτα με **in** — επόμενο βήμα.

Υπάρχει στη λίστα;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 6

```
guests = ["Άννα", "Νίκος", "Ελένη"]
print("Νίκος" in guests)
True
print("Μαρία" in guests)
False
```

ΜΕ IF

```
name = input("Όνομα: ")
if name in guests:
    print("Καλώς ήρθες!")
else:
    print("Δεν είσαι στη λίστα")
```

in

Απαντά **True** ή **False**: υπάρχει η τιμή μέσα στη λίστα; Είναι το ίδιο in που έψαχνε γράμμα σε κείμενο.

Αφαίρεση χωρίς σφάλμα

if "μέλι" in cart: και από κάτω **cart.remove("μέλι")**. Αν δεν υπάρχει, απλώς δεν γίνεται τίποτα.

Πόσες φορές;

Το **grades.count(10)** μετράει πόσες φορές υπάρχει το 10 στη λίστα.

ΣΥΝΔΕΣΗ Το if … in … είναι το ίδιο if του Μαθήματος 2. Η συνθήκη τώρα ρωτάει μια λίστα.

Τι έχει η λίστα μετά από κάθε γραμμή;

ΕΚΦΩΝΗΣΗ

Γράψε στο τετράδιο πώς είναι η λίστα **μετά από κάθε γραμμή**. Μετά τρέξε τον κώδικα στο «🧪 Ελεύθερο» του εργαστηρίου και έλεγξε.

ΛΥΣΗ

ΓΡΑΜΜΉ	Η ΛΊΣΤΑ ΜΕΤΆ	ΤΙ ΈΓΙΝΕ
`lst = [3, 1, 2]`	`[3, 1, 2]`	φτιάχτηκε
`lst.append(5)`	`[3, 1, 2, 5]`	5 στο τέλος
`lst.insert(0, 9)`	`[9, 3, 1, 2, 5]`	9 στη θέση 0
`lst[1] = 7`	`[9, 7, 1, 2, 5]`	η θέση 1 έγινε 7
`lst.remove(1)`	`[9, 7, 2, 5]`	έφυγε η τιμή 1
`x = lst.pop()`	`[9, 7, 2]`	x = 5

Η παγίδα

Το **remove(1)** βγάζει την **τιμή 1** — όχι το στοιχείο που είναι στη θέση 1 (εκείνη τη στιγμή το 7).

ΤΕΧΝΙΚΗ Μία γραμμή του πίνακα για κάθε εντολή — όπως ο πίνακας τιμών του Μαθήματος 2.

Περνάω από όλα τα στοιχεία με for

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 7

```
names = ["Άννα", "Νίκος", "Ελένη"]
for name in names:
    print("Γεια σου,", name)
Γεια σου, Άννα
Γεια σου, Νίκος
Γεια σου, Ελένη
```

Πώς δουλεύει

Σε κάθε γύρο η μεταβλητή **name** παίρνει το **επόμενο στοιχείο**. Όταν τελειώσουν τα στοιχεία, ο βρόχος σταματά μόνος του.

ΓΥΡΟ-ΓΥΡΟ

ΓΎΡΟΣ	NAME	ΤΥΠΏΝΕΙ
1ος	"Άννα"	Γεια σου, Άννα
2ος	"Νίκος"	Γεια σου, Νίκος
3ος	"Ελένη"	Γεια σου, Ελένη

Το όνομα το διαλέγεις εσύ

for name in names · **for g in grades** · **for x in nums**. Συνήθως: η λίστα στον πληθυντικό, η μεταβλητή στον ενικό.

ΙΔΙΟ ΜΕ ΠΡΙΝ To for ch in "ok" περνούσε από γράμματα. Το **for name in names** περνά από στοιχεία.

len · sum · max · min

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 8

grades = [12, 8.5, 17, 9, 15]

ΓΡΆΦΕΙΣ	ΠΑΊΡΝΕΙΣ	ΤΙ ΕΊΝΑΙ
`len(grades)`	`5`	πόσα στοιχεία
`sum(grades)`	`61.5`	άθροισμα
`max(grades)`	`17`	ο μεγαλύτερος
`min(grades)`	`8.5`	ο μικρότερος
`sum(grades) / len(grades)`	`12.3`	**μέσος όρος**

```
avg = sum(grades) / len(grades)
print(f"Μέσος όρος: {avg:.2f}")
Μέσος όρος: 12.30
```

Μέσος όρος

Δεν υπάρχει έτοιμη εντολή. Είναι πάντα **άθροισμα / πλήθος**, δηλαδή **sum / len**.

Και στις λέξεις

max(["Άννα", "Ελένη"]) → **'Ελένη'**: η τελευταία αλφαβητικά.

ΘΥΜΗΣΟΥ Το **:.2f** (Μάθημα 1) δείχνει 2 δεκαδικά, το **:.1f** ένα.

for + if: πόσοι; ποιοι;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 9 · 10

1 · ΜΕΤΡΗΤΗΣ: ΠΟΣΟΙ ΠΕΡΑΣΑΝ;

```
grades = [12, 8.5, 17, 9, 15]
count = 0
for g in grades:
    if g >= 10:
        count = count + 1
print("Πέρασαν:", count)
Πέρασαν: 3
```

2 · ΝΕΑ ΛΙΣΤΑ: ΠΟΙΟΙ ΠΕΡΑΣΑΝ;

```
grades = [12, 8.5, 17, 9, 15]
passed = []
for g in grades:
    if g >= 10:
        passed.append(g)
print(passed)
[12, 17, 15]
```

Μετρητής

Ξεκινά από **0 πριν** τον βρόχο. Μέσα, **+1** κάθε φορά που ισχύει το if. Όπως ο αθροιστής του Μαθήματος 2.

Νέα λίστα

Ξεκινά **άδεια [] πριν** τον βρόχο. Μέσα, **append** ό,τι περνά το if.

ΤΟ ΜΟΤΙΒΟ **Πριν** τον βρόχο: 0 ή []. **Μέσα**: if και πρόσθεση. **Μετά** τον βρόχο: print.

Γεμίζω τη λίστα από τον χρήστη

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 11 · 18

1 · ΞΕΡΩ ΠΟΣΕΣ ΤΙΜΕΣ: FOR

```
nums = []
for i in range(3):
    x = int(input("Αριθμός: "))
    nums.append(x)
print(nums)
Αριθμός: 4
Αριθμός: 7
Αριθμός: 1
[4, 7, 1]
```

2 · ΔΕΝ ΞΕΡΩ ΠΟΣΕΣ: WHILE

```
words = []
while True:
    w = input("Λέξη: ")
    if w == "":
        break
    words.append(w)
print(words)
Λέξη: γάτα
Λέξη: σκύλος
Λέξη:                # σκέτο Enter
['γάτα', 'σκύλος']
```

for

Όταν ξέρεις **πόσες** τιμές θα δοθούν (ή τις ρωτάς πρώτα: «Πόσοι μαθητές;»).

while True + break

Όταν σταματά ο χρήστης: πατά σκέτο **Enter** (κενή γραμμή) και το break βγαίνει από τον βρόχο.

ΣΥΧΝΟ ΛΑΘΟΣ Το **nums = []** μπαίνει **πριν** τον βρόχο. Αν το βάλεις μέσα, η λίστα αδειάζει σε κάθε γύρο!

Βάζω τη λίστα σε σειρά

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 12

NUMS.SORT() · ΑΛΛΑΖΕΙ ΤΗ ΛΙΣΤΑ

```
nums = [5, 2, 9, 1]
nums.sort()
print(nums)
[1, 2, 5, 9]
nums.sort(reverse=True)
print(nums)
[9, 5, 2, 1]
```

SORTED(NUMS) · ΦΤΙΑΧΝΕΙ ΝΕΑ

```
nums = [5, 2, 9, 1]
new = sorted(nums)
print(new)
[1, 2, 5, 9]
print(nums)
[5, 2, 9, 1]   # η παλιά έμεινε ίδια
```

reverse=True

Από τον **μεγαλύτερο** στον μικρότερο. Χωρίς αυτό: από τον μικρότερο στον μεγαλύτερο.

Λέξεις

Ταξινομούνται **αλφαβητικά**: sorted(["Νίκος", "Άννα"]) → ['Άννα', 'Νίκος'].

ΣΥΧΝΟ ΛΑΘΟΣ **nums = nums.sort()** → η λίστα γίνεται None (τίποτα). Το sort() γράφεται μόνο του.

Παίρνω ένα κομμάτι της λίστας

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 12

nums = [10, 20, 30, 40, 50, 60]

ΓΡΆΦΕΙΣ	ΠΑΊΡΝΕΙΣ	ΜΕ ΛΌΓΙΑ
`nums[0:3]`	`[10, 20, 30]`	από τη θέση 0 ως **πριν** την 3
`nums[:3]`	`[10, 20, 30]`	τα 3 πρώτα
`nums[2:5]`	`[30, 40, 50]`	θέσεις 2, 3, 4
`nums[3:]`	`[40, 50, 60]`	από τη θέση 3 ως το τέλος
`nums[-2:]`	`[50, 60]`	τα 2 τελευταία

Ο κανόνας

[αρχή:τέλος]: η αρχή μετράει, το **τέλος όχι**. Ίδιο με το range(αρχή, τέλος).

Τα 3 καλύτερα

Ταξινόμησε από τον μεγαλύτερο (βήμα 11) και πάρε το **[:3]**.

ΟΙ 3 ΚΑΛΥΤΕΡΟΙ

```
grades = [12, 17, 9, 19, 15]
top = sorted(grades, reverse=True)
print(top[:3])
[19, 17, 15]
```

ΚΑΙ ΣΤΑ ΚΕΙΜΕΝΑ Το **"Python"[:2]** δίνει "Py".

Θέση και τιμή μαζί

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 16

```
names  = ["Άννα", "Νίκος", "Ελένη"]
grades = [17, 9, 15]

for i in range(len(names)):
    print(names[i], grades[i])
Άννα 17
Νίκος 9
Ελένη 15
```

ΓΥΡΟ-ΓΥΡΟ

I	NAMES[I]	GRADES[I]
0	"Άννα"	17
1	"Νίκος"	9
2	"Ελένη"	15

range(len(names))

Δίνει τις θέσεις **0, 1, 2…**, όσα και τα στοιχεία. Με το **ίδιο i** παίρνεις από τις δύο λίστες το όνομα και τον βαθμό του ίδιου μαθητή.

Ποιος έχει τον μεγαλύτερο;

grades.index(17) → 0: σε ποια θέση είναι το 17.
Μετά **names[0]** → "Άννα".

ΠΡΟΣΟΧΗ Οι δύο λίστες πρέπει να έχουν **την ίδια σειρά** και **το ίδιο μήκος**.

Τα πιο συχνά λάθη με τις λίστες

ΤΙ ΈΓΡΑΨΕΣ	ΤΙ ΒΓΑΊΝΕΙ	Η ΔΙΌΡΘΩΣΗ
`fruits[4]  (με 4 στοιχεία)`	`IndexError`	οι θέσεις είναι 0–3· για το τελευταίο: **fruits[-1]**
`cart.remove("μέλι")`	`ValueError`	έλεγξε πρώτα: **if "μέλι" in cart:**
`nums = nums.sort()`	`None`	σκέτο **nums.sort()**
`nums = []  (μέσα στο for)`	`αδειάζει κάθε γύρο`	βάλε το **nums = []** πριν το for
`["μήλο" "αχλάδι"]`	`['μήλοαχλάδι']`	ξέχασες το **κόμμα**
`sum(x) / len(x)  (άδεια x)`	`ZeroDivisionError`	έλεγξε πρώτα: **if len(x) > 0:**

Διάβασε το μήνυμα του σφάλματος

Η **τελευταία γραμμή** λέει **τι** πήγε στραβά (π.χ. IndexError). Το **line N** λέει **σε ποια γραμμή**. Στο εργαστήριο, ο έλεγχος σου το εξηγεί και στα ελληνικά.

ΣΥΜΒΟΥΛΗ Όταν κάτι δεν βγαίνει, βάλε **print(λίστα)** μετά από κάθε βήμα και δες πώς αλλάζει.

Πλειάδα: τιμές που πάνε μαζί

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 13

```
point = (3, 4)        # θέση x, y
red = (255, 0, 0)     # χρώμα R, G, B

print(point[0])
3
x, y = point          # μοιράζω σε 2 μεταβλητές
print(x, y)
3 4
```

ΔΕΝ ΑΛΛΑΖΕΙ

```
point[0] = 10
TypeError: 'tuple' object does not support item assignment
```

Παρενθέσεις αντί για αγκύλες

Η πλειάδα (στα αγγλικά tuple) γράφεται με **( )**. Θέσεις, **len**, **in** και **for** δουλεύουν όπως στη λίστα.

Δεν αλλάζει

Δεν έχει append ούτε remove, ούτε αλλάζει θέση. Τη φτιάχνεις μία φορά και μένει έτσι.

x, y = point

Οι τιμές **μοιράζονται με τη σειρά** στις μεταβλητές αριστερά. Όσες τιμές, τόσες μεταβλητές.

PYGAME Στο pygame οι θέσεις είναι **(x, y)** και τα χρώματα **(R, G, B)**: screen.fill((255, 255, 255)).

Λίστα ή πλειάδα;

ΕΚΦΩΝΗΣΗ

Τι θα διάλεγες σε κάθε περίπτωση; Πρώτα σκέψου: θα **αλλάζει** ή μένει **ίδιο**;

ΛΥΣΗ

ΠΕΡΊΠΤΩΣΗ	ΔΙΑΛΈΓΩ	ΓΙΑΤΊ
α) Τα προϊόντα στο καλάθι ενός e-shop	**λίστα**	ο πελάτης βάζει και βγάζει
β) Η θέση (x, y) του παίκτη	**πλειάδα**	δύο τιμές που πάνε μαζί
γ) Ένα χρώμα RGB	**πλειάδα**	έτσι το θέλει το pygame
δ) Οι βαθμοί που δίνει ο χρήστης	**λίστα**	γεμίζει με append
ε) Οι 7 μέρες της εβδομάδας	**πλειάδα**	δεν αλλάζουν ποτέ

Ο γρήγορος κανόνας

Θα βάζεις ή θα βγάζεις στοιχεία; → **λίστα**.
Είναι ένα «πακέτο» που μένει ίδιο; → **πλειάδα**.

ΣΗΜΕΙΩΣΗ Και η λίστα θα «δούλευε» στα β, γ, ε. Η πλειάδα όμως **προστατεύει** τις τιμές από αλλαγή κατά λάθος.

Λίστα με λίστες: πίνακας

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Ασκήσεις 14 · 17

```
m = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 9]]

print(m[1])
[4, 5, 6]
print(m[1][2])
6
```

ΑΘΡΟΙΣΜΑ ΚΑΘΕ ΓΡΑΜΜΗΣ

```
for row in m:
    print(sum(row))
6
15
24
```

ΤΟ m ΣΑΝ ΠΙΝΑΚΑΣ

	ΣΤΗΛΗ 0	ΣΤΗΛΗ 1	ΣΤΗΛΗ 2
γραμμή 0	1	2	3
γραμμή 1	4	5	**6**
γραμμή 2	7	8	9

Δύο θέσεις

Πρώτα η **γραμμή**, μετά η **στήλη**: **m[γραμμή][στήλη]**. Το m[1][2] είναι η γραμμή 1, στήλη 2 → 6.

Πού θα το δεις

Τρίλιζα, σκακιέρα, η πίστα ενός παιχνιδιού 2D: όλα είναι πίνακες.

ΠΛΗΘΗ **len(m)** = πόσες γραμμές · **len(m[0])** = πόσες στήλες.

Οι 18 ασκήσεις, βήμα-βήμα

#	ΆΣΚΗΣΗ	ΕΠΊΠΕΔΟ	ΔΙΑΦΆΝΕΙΑ
1	Η πρώτη μου λίστα	Εύκολο	5
2	Πρώτη, τρίτη, τελευταία	Εύκολο	6
3	Διόρθωση βαθμού	Εύκολο	7
4	Καλάθι: βάζω προϊόντα	Εύκολο	8
5	Καλάθι: βγάζω προϊόντα	Εύκολο	9
6	Είσαι στη λίστα;	Εύκολο	10
7	Γεια σε όλους	Εύκολο	12
8	Θερμοκρασίες εβδομάδας	Μέτριο	13
9	Πόσοι πέρασαν;	Μέτριο	14

#	ΆΣΚΗΣΗ	ΕΠΊΠΕΔΟ	ΔΙΑΦΆΝΕΙΑ
10	Μόνο οι ζυγοί	Μέτριο	14
11	Γέμισε τη λίστα	Μέτριο	15
12	Οι 3 καλύτεροι	Μέτριο	16, 17
13	Πλειάδα: πόσο μακριά;	Μέτριο	20
14	Πίνακας: πόντοι ομάδων	Μέτριο	22
15	Βαθμολογίες τάξης	Δύσκολο	24
16	Πάνω από τον μέσο όρο	Δύσκολο	18, 25
17	Μαγικό τετράγωνο	Ο στόχος	22, 26
18	Λέξεις μέχρι κενή γραμμή	Για το σπίτι	15, 16

ΣΤΟ SITE Εργαστήριο → **Μάθημα 3**. Κάθε άσκηση έχει το «📖 Στις διαφάνειες» που ανοίγει κατευθείαν τη σωστή διαφάνεια.

Βαθμολογίες τάξης

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 15

ΕΚΦΩΝΗΣΗ

Διάβασε το πλήθος των μαθητών και τους βαθμούς τους. Εμφάνισε **μέσο όρο**, **μέγιστο** και **ελάχιστο**, **πόσοι πέρασαν** (≥ 10) και τους βαθμούς **ταξινομημένους** από τον μεγαλύτερο.

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
n = int(input("Πόσοι μαθητές; "))
grades = []
for i in range(n):
    g = float(input(f"Βαθμός {i + 1}: "))
    grades.append(g)

avg = sum(grades) / len(grades)
passed = 0
for g in grades:
    if g >= 10:
        passed = passed + 1

print(f"Μέσος όρος: {avg:.2f}")
print("Μέγιστος:", max(grades), " Ελάχιστος:", min(grades))
print("Πέρασαν:", passed, "από", n)
print("Ταξινομημένοι:", sorted(grades, reverse=True))
```

```
Πόσοι μαθητές; 5
Βαθμός 1: 12
Βαθμός 2: 8.5
...
Μέσος όρος: 12.30
Μέγιστος: 17.0  Ελάχιστος: 8.5
Πέρασαν: 3 από 5
Ταξινομημένοι: [17.0, 15.0, 12.0, 9.0, 8.5]
```

Από ποια βήματα;

10 γέμισμα · **8** sum, max, min · **9** μετρητής · **11** sorted

ΓΙΑΤΙ 17.0 Διαβάσαμε με **float()**, οπότε όλοι οι βαθμοί είναι δεκαδικοί και τυπώνονται με .0.

Ποιοι είναι πάνω από τον μέσο όρο;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 16

ΕΚΦΩΝΗΣΗ

Έχεις δύο λίστες: τα **ονόματα** και τους **βαθμούς** (στην ίδια σειρά). Εμφάνισε τον μέσο όρο, όσους είναι **πάνω** από αυτόν και το όνομα του **καλύτερου**.

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
names = ["Άννα", "Νίκος", "Μαρία", "Γιώργος", "Ελένη"]
grades = [12, 8.5, 17, 9, 15]

avg = sum(grades) / len(grades)
print(f"Μέσος όρος: {avg:.2f}")

for i in range(len(names)):
    if grades[i] > avg:
        print(" ", names[i], grades[i])

best = grades.index(max(grades))
print("Καλύτερος/η:", names[best])
```

```
Μέσος όρος: 12.30
  Μαρία 17
  Ελένη 15
Καλύτερος/η: Μαρία
```

Η γραμμή του best

max(grades) → **17** · grades.index(17) → **2** · names[2] → **Μαρία**

Από ποια βήματα;

8 sum, len, max · **13** δύο λίστες με range(len(...))

ΣΥΜΒΟΥΛΗ Κόλλησες; Λύσε πρώτα μόνο τον μέσο όρο, μετά μόνο τη λίστα, και στο τέλος τον καλύτερο.

Είναι μαγικό τετράγωνο;

ΣΤΟ ΕΡΓΑΣΤΗΡΙΟ
Άσκηση 17

ΕΚΦΩΝΗΣΗ

Ένας πίνακας 3×3 είναι **μαγικός** όταν όλες οι γραμμές, όλες οι στήλες και οι δύο διαγώνιοι έχουν **το ίδιο άθροισμα**. Μάζεψε τα αθροίσματα σε μια λίστα και πες αν είναι μαγικός.

ΛΥΣΗ · ΔΟΚΙΜΑΣΕ ΠΡΩΤΑ ΜΟΝΟΣ/Η ΣΟΥ

```
m = [[2, 7, 6],
     [9, 5, 1],
     [4, 3, 8]]

sums = []
for row in m:                           # γραμμές
    sums.append(sum(row))
for c in range(3):                      # στήλες
    sums.append(m[0][c] + m[1][c] + m[2][c])
sums.append(m[0][0] + m[1][1] + m[2][2])   # διαγώνιος ↘
sums.append(m[0][2] + m[1][1] + m[2][0])   # διαγώνιος ↙

print("Αθροίσματα:", sums)
if min(sums) == max(sums):
    print("Μαγικό!")
else:
    print("Όχι μαγικό")
```

```
Αθροίσματα: [15, 15, 15, 15, 15, 15, 15, 15]
Μαγικό!
```

Η ιδέα

Αν ο **μικρότερος** από τα αθροίσματα είναι ίσος με τον **μεγαλύτερο**, τότε είναι **όλα ίδια**.

Από ποια βήματα;

15 πίνακας · **4** append · **8** sum, min, max

ΠΡΟΚΛΗΣΗ Άλλαξε το 5 σε 6 και τρέξε ξανά. Τι βγαίνει; Γιατί;

Τι κρατάμε

ΘΈΛΩ ΝΑ…	ΓΡΆΦΩ
φτιάξω λίστα	`nums = [4, 7, 1]`
πάρω το πρώτο / το τελευταίο	`nums[0]` · `nums[-1]`
αλλάξω ένα στοιχείο	`nums[1] = 9`
προσθέσω στο τέλος	`nums.append(5)`
βγάλω μια τιμή	`nums.remove(7)` · `nums.pop()`
δω αν υπάρχει	`if 7 in nums:`
περάσω από όλα	`for x in nums:`
βρω πλήθος, άθροισμα, max, min	`len` · `sum` · `max` · `min`
ταξινομήσω	`nums.sort()` · `sorted(nums)`
πάρω τα 3 πρώτα	`nums[:3]`

Πλειάδα ( )

Σαν λίστα που **δεν αλλάζει**. Για τιμές που πάνε μαζί: **(x, y)**, **(R, G, B)**. x, y = point.

Επόμενη φορά · Δευ 26/10

Σύνολα και λεξικά: συλλογές χωρίς διπλότυπα και αναζήτηση με «κλειδί», π.χ. όνομα → τηλέφωνο.

ΓΙΑ ΤΟ ΣΠΙΤΙ Άσκηση **18** στο εργαστήριο: διάβαζε λέξεις μέχρι κενή γραμμή (βήμα 10).